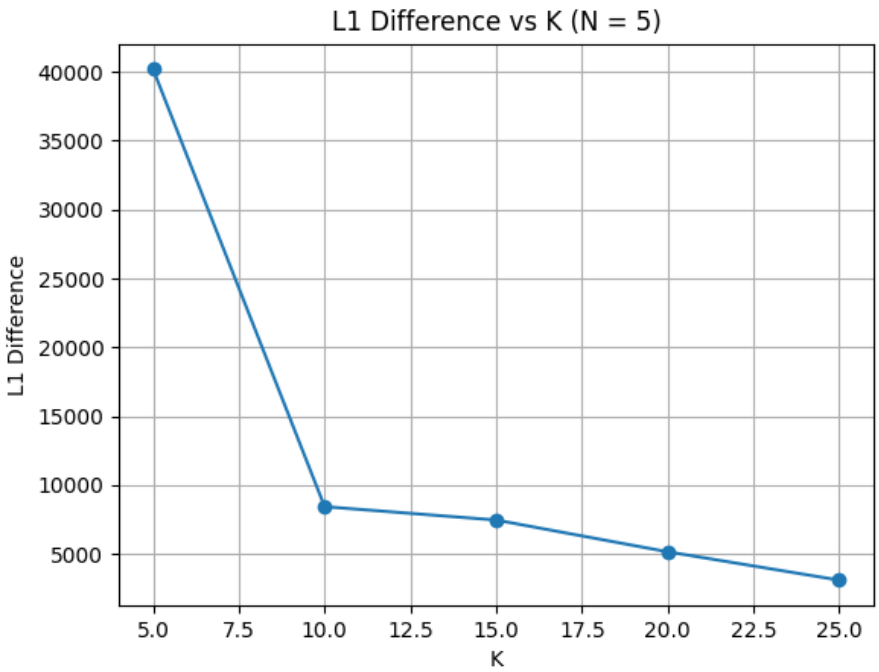
Question 3

(a) Kernel Estimation Function

The code estimates the transition kernel $P(s' \mid s, a)$using the simulator instead of computing it analytically. For each state-action pair, the simulator is called Ktimes to observe the next state transitions. A count matrix is maintained to record how many times each next state is reached. The indexing row_idx $= s \times A + a$is used to map each (state, action) pair to a unique row, and the resulting next states are mapped to column indices using a state-to-index mapping. After collecting counts for all Ksamples, the counts are normalized by dividing by K, converting them into probabilities. This produces an empirical estimate of the kernel where each row approximates the true transition probability distribution.

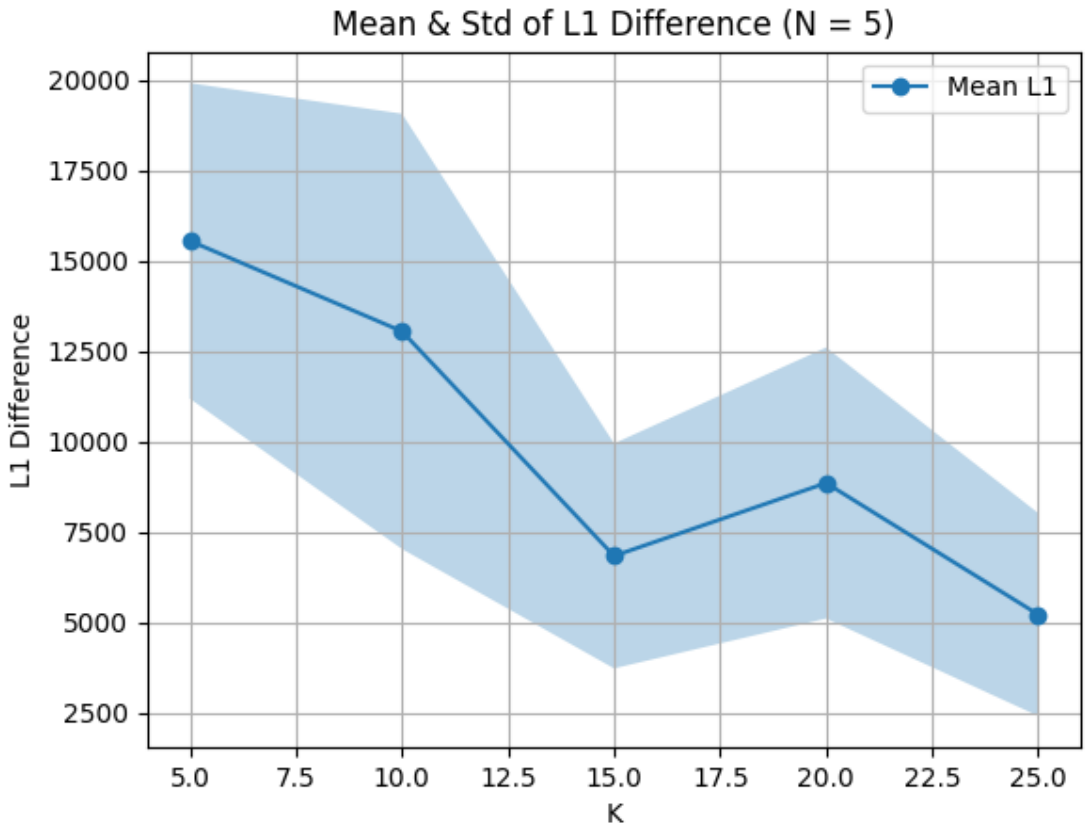
(b) L1 Difference vs K

In this part, the estimated kernel is used to compute the Q-function using value iteration for different values of K. The resulting Q-function is then compared with the optimal Q-function obtained using the exact kernel. The difference is measured using the L1 norm, defined as the sum of absolute differences between corresponding Q-values. As Kincreases, the estimated kernel becomes more accurate, and the L1 difference decreases. This demonstrates that increasing the number of simulator samples improves the quality of the approximation.



(c) Mean and Standard Deviation Analysis

To account for randomness in the simulator, the kernel estimation and Q-function computation are repeated for multiple independent runs using different random seeds. For each value of K, the L1 difference is computed across all runs, and the mean and standard deviation are calculated. The mean represents the average estimation error, while the standard deviation captures the variability due to randomness. These values are plotted as a function of K, typically showing a decreasing trend in both mean and variability as K increases, indicating improved accuracy and stability of the estimated kernel.



Naming Conventions and Implementation Details

Naming Conventions

A clear and consistent naming convention has been followed to maintain readability and direct correspondence with the problem statement and mathematical formulation.

All files follow the format:
Q3<function and question number details>.py

Examples:

- Q3a_estimate_kernel_func.py → Kernel estimation using simulator
- Q3b_evaluate_policy.py → L1 difference computation
- Q3c_mean_stdDev_of_L1.py → Mean and standard deviation analysis

Function names use **snake_case** and clearly describe their purpose:

- estimate_kernel → estimates transition probabilities using simulator
- compute_L1_difference → computes difference between exact and estimated Q
- compute_mean_std → computes mean and standard deviation over multiple runs

Variable Naming

Variable names are aligned with reinforcement learning notation:

- **S** → Number of states
- **A** → Number of actions
- **K** → Number of simulator samples per (state, action)
- **s_idx** → Index of current state
- **a_idx** → Index of current action
- **row_idx = s * A + a** → Maps (state, action) to kernel row
- **col_idx** → Index of next state
- **P_sa** → Transition probability vector
- **Q_exact** → Q-function from exact kernel
- **Q_estimated** → Q-function from estimated kernel
- **L1** → Sum of absolute differences between Q-functions

This ensures the code directly reflects the mathematical equations used in the analysis.

Implementation Details

The implementation follows a modular approach, where each part of the question is implemented in a separate file. The kernel estimation function replaces analytical computation with sampling by calling the simulator multiple times for each (state, action) pair. The observed transitions are stored as counts and then normalized to form probability distributions, ensuring that each row of the estimated kernel sums to 1.

The value iteration function from Q2 is reused to compute Q-functions for both exact and estimated kernels, ensuring consistency in comparison. The L1 difference is computed by summing the absolute differences between corresponding entries of the two Q-functions.

For statistical analysis, the entire process is repeated multiple times with different random seeds. The mean and standard deviation are computed across these runs to capture both accuracy and variability of the estimation.

Matrix operations such as normalization and dot products are used to improve computational efficiency. The use of indexing row_idx $= s \times A + a$ensures efficient mapping between (state, action) pairs and kernel rows. Overall, the implementation closely follows the theoretical framework while maintaining clarity, modularity, and scalability.